



Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

Heuristic initialization of PSO task scheduling algorithm in cloud computing

Seema A. Alsaidy^a, Amenah D. Abbood^a, Mouayad A. Sahib^{b,*}^a University of Baghdad, College of Science, Computer Science Dept., Baghdad, Iraq^b University of Information Technology and Communications, College of Engineering, Baghdad, Iraq

ARTICLE INFO

Article history:

Received 21 July 2020

Revised 3 November 2020

Accepted 4 November 2020

Available online 13 November 2020

Keywords:

Particle swarm optimization

Task scheduling

Cloud computing

Metaheuristic algorithms

Load balancing

Virtual machines

ABSTRACT

Task scheduling is one of the major issues in cloud computing environment. Efficient task scheduling is substantial to attain cost-effective execution and improve resource utilization. The task scheduling problem is classified to be a nondeterministic polynomial time (NP)-hard problem. This feature attracts researchers to utilize nature inspired metaheuristic algorithms. Initializing searching solutions randomly is one of the key features in such optimization algorithms. However, assisting metaheuristic algorithms with effective initialized solutions can significantly improve its performance. In this paper, an improved initialization of particle swarm optimization (PSO) using heuristic algorithms is proposed. Longest job to fastest processor (LJFP) and minimum completion time (MCT) algorithms are used to initialize the PSO. The performance of the proposed LJFP-PSO and MCT-PSO algorithms are evaluated in minimizing the makespan, total execution time, degree of imbalance, and total energy consumption metrics. Moreover, the performance of the proposed algorithms is compared with recent task scheduling methods. Simulation results revealed the effectiveness and superiority of the proposed LJFP-PSO and MCT-PSO compared to the conventional PSO and comparative algorithms.

© 2020 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

The tremendous increase of internet information processing lead to the development of cloud computing system. Cloud computing plays an important role in offering technology services through the internet. It provides users with computer system resources, such as data storage and computing power, without direct active management. A cloud can provide three types of services related to the infrastructure, platform, and software. The first service is infrastructure as a service (IaaS) which provides infrastructure services such as storage system and computation resources. Second service is platform as a service (PaaS), which enables clients to build their applications on the provided platform. The third service provides the software to the users termed

software as a service (SaaS), in which users can use the software directly from the cloud without the need of installing it locally (Devaraj et al., 2020). Cloud providers exploit virtualization technology and supply users with computing resources in the form of virtual machines (VMs). When multiple users request services from the cloud in the form of tasks, an efficient task scheduling is required to improve the overall performance of the cloud computing. Efficient task scheduling allows optimal allocation of resources among the requested tasks in a finite time to achieve desired quality of service (QoS) (Adhikari et al., 2019). The problem of optimal scheduling is to build a schedule of tasks to VMs allocation subject to some constraints to optimize a given objective. Therefore, the task scheduling algorithm is one of the core elements of each cloud infrastructure.

In cloud computing, the main types of scheduling mechanisms are cloud service scheduling, heuristics scheduling, workflow scheduling, static scheduling and dynamic scheduling (Arunarani et al., 2019). However, scheduling techniques can be classified based on the complexity of the algorithm as heuristic, metaheuristic, and hybrid task scheduling approaches (Elaziz et al., 2019). Heuristic algorithms are used for static scheduling such algorithms include minimum execution time (MET), minimum completion time (MCT), shortest job to fastest processor (SJFP),

* Corresponding author.

E-mail addresses: seema.alsaidy@gmail.com (S.A. Alsaidy), amenah.abbood@sc.uobaghdad.edu.iq (A.D. Abbood), mouayad.sahib@uoitc.edu.iq (M.A. Sahib).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

longest job to fastest processor (LJFP), Min-Min, Max-Min (Elaziz et al., 2019; Mishra et al., 2020). The task scheduling problem is defined to be a non-deterministic polynomial (NP)-hard optimization problem (Golchi et al., 2019). Therefore, the need for metaheuristic algorithms is necessary to deal with such multimodal optimization problems. The main metaheuristic algorithms used are particle swarm optimization (PSO) (Mansouri et al., 2019; Kumar and Sharma, 2018), ant colony optimization (ACO) (Mahato et al., 2017), genetic algorithm (Keshanchi et al., 2017), discrete symbiotic organism search (DSOS) (Abdullahi et al., 2016), gravitational search algorithm (GSA) (Chaudhary and Kumar, 2018).

In order to obtain improved performance, researchers used hybrid metaheuristic algorithms. Examples of such hybridization include; firefly and PSO (Golchi et al., 2019; Devaraj et al., 2020), Q-learning and PSO (Jena et al., 2020), firefly and simulated annealing (Fanian et al., 2018), hybridized whale optimization algorithm (Strumberger et al., 2019).

In using PSO for task scheduling, the particles are initialized randomly by allocating each task to one of the available machines without constraints. Random initialization is one of the key features of PSO in spreading the particles over the search space. However, if these random solutions are chosen wisely, the searching process of tasks to VMs allocation can be boosted to the optimal solution. Moreover, heuristic algorithms such as MCT, SJFP, LJFP, Min-Min, and Max-Min are non-optimized basic algorithms. However, compared to metaheuristic algorithms, they are simple to implement. If the PSO can be assisted by suggesting a good starting search point, the performance of the algorithm can be improved.

In this paper, the proposed method is to utilize heuristic scheduling algorithms to initialize the PSO search process. The two standard algorithms are LJFP and MCT. The proposed combined algorithms termed LJFP-PSO and MCT-PSO are expected to improve the classical PSO.

The major contributions of this paper can be summarized as follows:

1. Formulating the problem of task scheduling and defining the mathematical model and the objective functions for which the allocation of tasks to VMs is optimized.
2. Proposing a hybridized task scheduling algorithm and designing it by replacing the random initialization of the metaheuristic algorithm (selected to be PSO algorithm) using heuristic algorithms (selected to be MCT and LJFP algorithms) termed as MCT-PSO and LJFP-PSO.
3. Evaluating and comparing the performance of the proposed MCT-PSO and LJFP-PSO with four recent and well-known task scheduling algorithms. The evaluation is performed using four objective metrics which are makespan (MS), total execution time (TET), degree of imbalance (DI), and total energy consumption (TEC).

The rest of the paper is organized as follows. In Section 2, significant related works are presented. The problem of task scheduling in cloud computing environment is described and modelled in section 3. The proposed algorithms are explained in Section 4. Simulation results are discussed in Section 5. Finally, Section 6 concludes the paper.

2. Related work

Researchers attention and concern has been attracted by many issues facing cloud computing. Such major issues include resource management, load balancing, migration to clouds, privacy and security, energy-efficiency, availability and scalability, interoperability and compatibility (Al-Arasi and Saif, 2020). These

issues are significantly affected by exploring an optimal task scheduling. A scheduling model in the cloud environment is shown in Fig. 1.

The datacenter broker (DB) is responsible for detecting and gathering all information related to the available resources (VMs) and all remaining resources that may be available in future. The DB collect this information with the help of cloud information services (CIS). The hypervisor or virtual machine monitor (VMM) acts as an interface between the host operating system and VMs. Tasks are submitted to a task waiting queue to be scheduled on VMs based on a defined scheduling algorithm in DB.

The problem of scheduling involves scheduling a given set of tasks on a given set of resources in the form of virtual machines subject to some constraints to optimize some objective function. In cloud computing, the effectiveness of task scheduling is measured by various system performance parameters (metrics). Generally, these optimization criteria are categorized into two demands based on cloud service: cloud users demand and cloud service providers demand. On one hand, makespan (MS) and total execution time (TET) are user demand criteria. On the other hand, degree of imbalance (DI) and energy consumption are provider demand criteria. In single objective task scheduling, only one criterion is considered, however, in multi-objective criteria, two or more parameters are considered as an objective (Belgacem et al., 2020; Devaraj et al., 2020).

In recent years, a considerable amount of literature has been published on task scheduling in cloud computing environment. These studies can be categorized into two main groups, heuristic or metaheuristic algorithms, and hybrid of heuristic and metaheuristic algorithms. (Kaur and Kaur, 2022) proposed two heuristic metaheuristic hybridized approaches to optimize the load balancing in cloud environment. The two heuristic algorithms are predicted earliest finish time (PEFT) and heterogeneous earliest finish time (HEFT). The output of these two heuristics are employed to initialize the ACO algorithm parameters to speed up the convergence rate and reduce randomness. (Golchi et al., 2019) proposed a hybrid of firefly and improved particle swarm optimization (IPSO) algorithm for load balancing and tasks scheduling in cloud computing environment. The random nature of the operators of the IPSO algorithm make it sensitive to the initial population. Improper selection of initial population significantly effects the algorithm's convergence. Therefore, firefly algorithm is utilized to initialize the IPSO. (Belgacem et al., 2020) proposed a multi-objective search algorithm called spacing multi-objective antlion algorithm (S-MOAL) then (Abualigah and Diabat, 2020) proposed a hybrid multi-objective antlion optimization algorithm (MALO) with elite-based differential evolution (DE) for solving task scheduling problems in cloud computing environments. The DE is used to improve the exploitation search-ability of the ALO since the DE algorithm demonstrates the powerful features of the genetic algorithm (GA) and the evolution strategy (ES). (Elaziz et al., 2019) also used DE algorithm as a local search technique to enhance the exploitation ability of standard moth search algorithm (MSA) and termed it as (MSDE). The MSDE is used to balance the load of tasks among VMs. (Strumberger et al., 2019) introduced a hybridized whale optimization algorithm (WOA) for tackling the resource scheduling problem. The enhanced WOA is improved by performing a hybridization with the artificial bee colony (ABC) and firefly algorithms (FA) to balance between exploitation (intensification) and exploration (diversification) mechanisms.

The main research works related to this paper are listed and compared in Table 1.

As can be seen from Table 1, the works are based on different types of metaheuristic algorithms. However, the common relationship between these related approaches are the use of random population in initializing the metaheuristic and hybrid metaheuristic

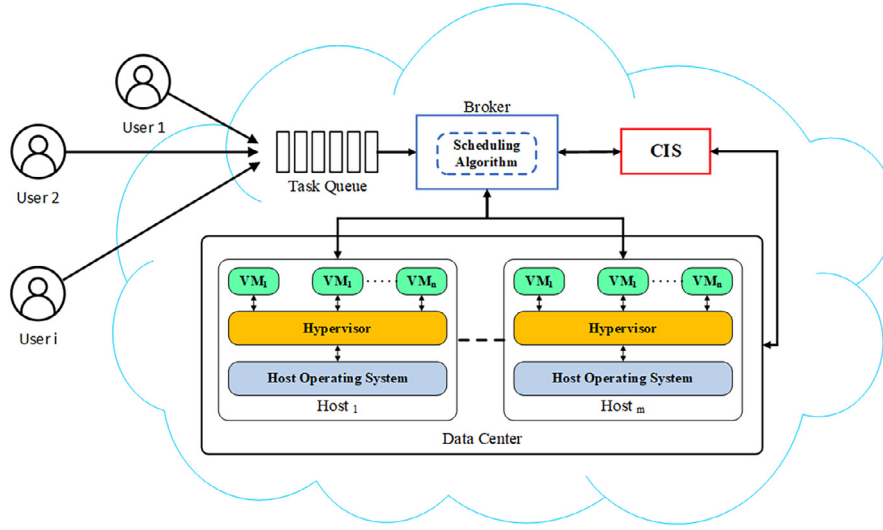


Fig. 1. Scheduling model in cloud environment.

algorithms. Generally, the convergence of metaheuristic algorithms is significantly affected by the initial population. The random nature is a fundamental need to avoid local minimum traps. However, if some particles are assisted heuristically in selecting an effective or near optimum starting points, then the algorithm convergence can be improved. In this paper, the idea of the proposed algorithm is to utilize heuristic algorithms for initializing the population and hence improve the algorithm's performance significantly.

3. Task scheduling problem formulation

In cloud computing, tasks are scheduled based on the objective of improving the performance of various quality of service parameters. The problem of task scheduling is how to find the optimal allocation of a number of tasks to a given number of virtual processing machines. To model this problem, the following assumptions are considered:

- 1- All submitted tasks are independent.
- 2- Tasks cannot migrate between virtual machines, in other words, one task is not allowed to be assigned to more than one virtual machine.
- 3- The virtual machines are heterogeneous having different processing abilities and different power consumptions.
- 4- The VMM allows the optimization of the resources utilization with a minimum energy consumption.

Consider N_t independent tasks defined by a task length set, $T = \{T_1, T_2, \dots, T_{N_t}\}$, where T_i is the i -th task length in million instructions (MI). Each task in T , is bounded by T_{max} and T_{min} , i. e., $T_{min} \leq T_i \leq T_{max}$.

Also, consider N_v virtual machines defined by execution rate set, $V = \{V_1, V_2, \dots, V_{N_v}\}$, where V_j is the j -th virtual machine execution rate in million instructions per second (MIPS). Each virtual machine in V , is bounded by V_{max} and V_{min} , i. e., $V_{min} \leq V_j \leq V_{max}$. Similarly, we define the power set $P = \{P_1, P_2, \dots, P_{N_v}\}$, where P_j is the j -th virtual machine power consumption in watt bounded by P_{max} and P_{min} . The estimated computation time for executing task i on virtual machine j is given by:

$$ECT_{ij} = \frac{T_i}{V_j} \quad (1)$$

The ultimate output of the scheduling algorithm is an $N_t \times N_v$ allocation matrix stating which task should be executed by which VM. The allocation matrix is defined to be:

$$X = \begin{bmatrix} x_{11} & \dots & x_{1N_v} \\ \vdots & \ddots & \vdots \\ x_{N_t1} & \dots & x_{N_tN_v} \end{bmatrix}, \text{ where } x_{ij} = \begin{cases} 1 & \text{if } T_i \text{ is allocated to } V_j \\ 0 & \text{if } T_i \text{ is not allocated to } V_j \end{cases} \quad (2)$$

With the condition:

$$\sum_{j=1}^{N_v} x_{ij} = 1 \text{ For } 1 \leq i \leq N_t \quad (3)$$

Thus, the j -th virtual machine execution time (VET_j) is given by:

$$VET_j = \sum_{i=1}^{N_t} x_{ij} * ECT_{ij} \text{ For } 1 \leq j \leq N_v \quad (4)$$

To evaluate the performance of a given scheduling algorithm, four metrics will be considered; MS, TET, DI, and TEC which are given in the following equations respectively:

$$MS = \max\{VET_j\} \text{ For } 1 \leq j \leq N_v \quad (5)$$

$$TET = \sum_{j=1}^{N_v} VET_j \text{ For } 1 \leq j \leq N_v \quad (6)$$

$$DI = N_v \left[\frac{MS - VET_{min}}{TET} \right] \quad (7)$$

Where VET_{min} is the minimum total execution time between all machines. During the execution of tasks, a VM can have one of two states; active and idle states. The idle state of a VM consumes 60–70% of energy consumption of active state of that VM (Jena et al., 2020; Mishra et al., 2020). Accordingly, assume virtual machine j consumes energy equal to α_j (joule/MI) in active state and β_j (joule/MI) in idle state. A VM j reside (VET_j) sec. in active state and $(MS - VET_j)$ sec. in idle state, thus, the total energy consumption can be given as:

$$TEC = \sum_{j=1}^{N_v} [(VET_j * \alpha_j + (MS - VET_j) * \beta_j) * V_j] \quad (8)$$

The allocation of tasks stated by the scheduling algorithms are optimized based on the aforementioned metrics. The following section present the method of the proposed scheduling algorithm along with various standard ones.

Table 1
Comparison of related work.

	References	Metrics	Proposed Algorithm	Approaches	Parameters Variation	Simulation Environment
1	(Abualigah and Diabat, 2020) Compared with	Makespan Resource utilization	Hybrid antlion optimization algorithm with elite-based differential evolution (MALO)	Multi objective hybrid algorithms	Number of Tasks	CloudSim
2	(Jena et al., 2020) Compared with	Makespan, Energy utilization, Throughput, Degree of imbalance Total execution time	Hybridized modified PSO (MPSO) and improved Q-learning algorithm (QMPSO)	Adjust the velocity of the MPSO through the gbest and pbest based on the best action generated through the improved Q-learning	Number of Tasks Number of VMs Processing power	CloudSim
3	(Mishra et al., 2020) Compared with	Makespan, Energy Consumption	Comparison of Modern load Balancing Algorithms	Load Balancing	Number of Tasks Number of VMs	CloudSim
4	(Devaraj et al., 2020) Compared with	Execution time, Resource Utilization, Reliability, Make span and Average Throughput.	Hybrid of firefly and Improved Multi-Objective Particle Swarm Optimization technique. (FIMPSO)	Multi objective hybrid algorithms.	Number of Tasks. Tasks size.	MATLAB Tool
5	(Belgacem et al., 2020) Compared with	Weighted Round Robin (WRR), Random Deployment (RD), Dynamic load Balancing Approach (DLB) Load Balancing based on Bayes and Clustering (LB-BC) Makespan, Deadline, Cost, and Energy consumption	Spacing multi-objective antlion algorithm (S-MOAL).	Roulette wheel operator for parameter selection	Number of Tasks.	CloudSim
6	(Kaur and Kaur, 2022) Compared with	Makespan and cost. HPA and HHA	Hybrid Predict Earliest Finish Time (PEFT) heuristic with ACO (HPA), and Hybrid Heterogeneous Earliest Finish Time (HEFT) heuristic with ACO (HHA).	Hybridization of heuristic techniques with metaheuristic algorithm.	Number of VMs	Cloud workflow Simulator (CWS)
7	(Elaziz et al., 2019) Compared with	Makespan and Degree of imbalance PSO, WOA, MSA, shortest job first (SJF) and Round Robin (RR).	Improvement MSA using DE (MSDE)	Improve exploitation ability of MSA using DE	Number of Tasks	CloudSim
8	(Strumberger et al., 2019) Compared with	Makespan and cost. PSO, MSDE, Min-Min, FCFS, PBACO, CPSO, WOA	Hybridized WOA with ABC and FA	Balance between exploitation and exploration mechanisms	Number of Tasks	CloudSim
9	(Kumar and Sharma, 2018) Compared with	Makespan, Throughput, Cost, and Energy Consumption PSO, Honey Bee and Min-Min	PSO based cost and energy efficient scheduling (PSO-COGEN)	Trade-off between profit and energy consumption	Number of Tasks Number of VMs	CloudSim
10	(Chaudhary and Kumar, 2018) Compared with	Transfer Time and Total Cost Segmented Min-Min, Genetic Simulated Annealing, Simulated Annealing (SA), GA, Tabu Search, Min-Min, FCFS and PSO	Cloudy Gravitational Search Algorithm (Cloudy-GSA)	Newton's law of gravity is used for scheduling of load	Data vectors	CloudSim
11	(Abdullahi et al., 2016) Compared with	Makespan Response time and Degree of imbalance PSO	Discrete Symbiotic Organism Search (DSOS)	Develop discrete version of SOS algorithm	Number of Tasks	CloudSim

4. Proposed scheduling algorithm

Particle swarm optimization is one of the main meta-heuristic techniques utilized in solving optimal task scheduling problem. PSO is an evolutionary computational technique motivated by social behavior of animal species living in large colonies like birds, ants or fish (Sardaraz and Tahir, 2019). The mathematical modelling of the PSO algorithm is based on a number of agents grouped in a swarm called particles. Each particle is considered as a solution to the optimization problem. The particles are utilized to search for

an optimal solution (best particle position) within a predefined search space. The particle cooperates between each other sharing information about their current situation in the search process to update their new position. The update equation of the new particle position is given by:

$$p_l^{t+1} = p_l^t + v_l^{t+1} \quad (9)$$

where p_l^t is the l -th particle at iteration t , and v_l^{t+1} is the velocity of the l -th particle at iteration $t + 1$ defined by:

$$v_l^{t+1} = \omega v_l^t + c_1 r_1 (p_l^{best} - p_l^t) + c_2 r_2 (p^{gbest} - p_l^t) \quad (10)$$

Where p_l^{best} is the best position found so far associated with particle l . p^{gbest} is the best global position found among all p_l^{best} for $(1 \leq l \leq L)$, where L is the population size (number of particles). c_1 and c_2 are the acceleration constants, r_1 and r_2 are random numbers between 0 and 1, ω is the inertia factor. The velocity equation is influenced by three components; inertia, personal influence, and social influence represented by v_l^t , p_l^{best} , and p^{gbest} respectively.

For the task scheduling problem, each particle is structured by an allocation matrix as given by equation (up) and shown in Fig. 2. Therefore, the swarm is a three-dimensional matrix of size $(N_t \times N_v \times L)$.

The allocation matrix has two main features; First, all elements have a binary value of 0 or 1 ($x_{ij} \in \{0, 1\}$), second, for a given particle, each task allocation row contains one element equal to one and the rest are 0. For example, as shown in Fig. 2, particle 1 (p_1) assigns task 1 (T_1) to machine 1 (V_1), task 2 (T_2) to machine N_v (V_{N_v}), and task N_t (T_{N_t}) to machine 2 (V_2).

Similarly, the velocity 3D matrix will have the same size of swarm such that each velocity window of size $(N_t \times N_v)$ is associated with one particle. To ensure convergence of algorithm, the velocity matrix elements are bounded by the interval $[v_{min}, v_{max}]$.

In the conventional PSO, the particles are updated according to Eq. (9). However, in the problem of task scheduling where particles are defined with binary values, the particles update equation is derived based on the binary PSO as follows; for each velocity of particle l , get the index j_{max} of the machine at which the velocity is maximum compared to all values in the i -th row. Set the corresponding element of particle l at location (j_{max}, i) to '1' and all other locations at $(j, i)_{j \neq j_{max}}$ to '0'. As shown in Fig. 3, for example, for task 1, the maximum velocity appears at location $(i = 1, j_{max} = 2)$ with a value equal to $v_{1,2}^1$, thus task 1 will be allocated to VM 2 by setting $x_{12} = 1$ and the rest values of the row are set to 0.

In PSO, the particles are initialized randomly by allocating each task randomly to one of the available machines without constraints. Random initialization is one of the key features of PSO in spreading the particles over the search space. However, if we could select a good starting search point the performance of the algorithm can be improved. Therefore, the proposed method is to utilize heuristic scheduling algorithms to initialize the PSO search process. The two standard algorithms are LJFP and MCT. The proposed combined algorithms termed LJFP-PSO and MCT-PSO are

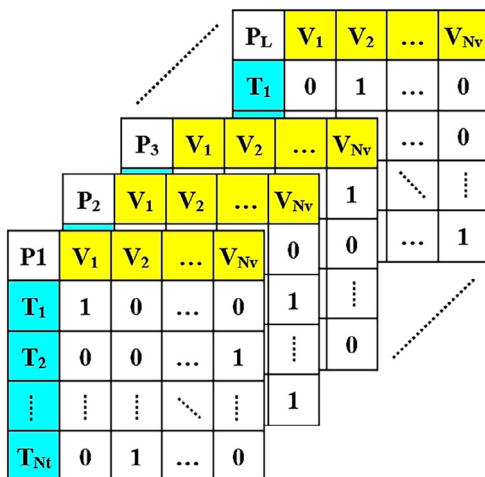


Fig. 2. Particles swarm structure.

expected to improve the classical PSO. The pseudo code of the LJFP, MCT, and the Proposed LJFP-PSO and MCT-PSO are presented in algorithms 1, 2, and 3 respectively.

Algorithm 1: Pseudo-code of MCT algorithm

```

1  Input:
2  No. of tasks ( $N_t$ ),
3  Task lengths ( $T$ ),
4  No. of VMs ( $N_v$ ),
5  VMs execution rates ( $V$ )
6  Output:
7  Allocation matrix (task to VM) of size  $N_t \times N_v$ 
8  Start:
9  Calculate the estimated execution time matrix of size
    $N_t \times N_v$ 
10 Initialize the allocation matrix (task to VM) of size  $N_t \times N_v$ 
   to zeros
11 For  $i = 1$  to no. of tasks  $N_t$ 
12   Find the minimum execution time of task  $i$  among all VMs
13   Set  $Mn = \text{minimum execution time}$ ,
14   Set  $j_{min} = \text{the index of the VM at which the minimum found}$ 
15   Allocate task  $i$  to VM at index  $j_{min}$  in the allocation matrix
16   Add  $Mn$  to all values of column  $j_{min}$  in the execution time
   matrix
17 End

```

In algorithm 1, the main loop of the MCT starts at step 11 and ends at step 17. This loop will run for N_t iterations equal to the number of tasks. At each iteration i , the minimum execution time of task i at VM j is found. Then in step 15, task i , is assigned to VM j , then, in step 16, the minimum execution time found is added to all tasks with index $(i + 1)$ to N_t and the process repeats until the loop ends.

Algorithm 2: Pseudo-code of LJFP algorithm

```

1  Input:
2  No. of tasks ( $N_t$ ),
3  Task lengths ( $T$ ),
4  No. of VMs ( $N_v$ ),
5  VMs execution rates ( $V$ )
6  Output:
7  Allocation matrix (task to VM) of size  $N_t \times N_v$ 
8  Start:
9  Calculate the estimated execution time matrix of size
    $N_t \times N_v$ 
10 Initialize the allocation matrix (task to VM) of size  $N_t \times N_v$ 
   to zeros
11 Sort the task according to their lengths in descending order
12 Sort the VMs according to their execution rate in descending
   order
13 For  $i = 1$  to no. of tasks  $N_t$ 
14   If  $\text{mod}(i, N_v) = 0$ 
15     VM index  $j = N_v$ 
16   else
17     VM index  $j = \text{mod}(i, N_v)$ 
18   end
19   Allocate task  $i$  to VM at index  $j$  in the allocation matrix
20 End

```

The LJFP as explained in algorithm 2, starts by Sorting the tasks and VMs according to their lengths (ML) and execution rate (MIPS) in descending order respectively. Similar to MCT algorithm, the main

Updated Velocity v_i^{t+1}					New Particle p_i^{t+1}				
v_i^{t+1}	V_1	V_2	...	V_{N_v}	p_i^{t+1}	V_1	V_2	...	V_{N_v}
T_1	$v_{1,1}$	$v_{1,2}^*$	...	v_{1,N_v}	T_1	0	1	...	0
T_2	$v_{2,1}$	$v_{2,2}$	...	v_{2,N_v}^*	T_2	0	0	...	1
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
T_{N_t}	$v_{N_t,1}^*$	$v_{N_t,2}$	...	v_{N_t,N_v}	T_{N_t}	1	0	...	0

Fig. 3. Method of particles update.

loop of the LJFP run for N_t iterations equal to the number of tasks from step 13–20. In each iteration, the sorted task is assigned to its corresponding sorted VM. The assignment process repeats in cyclic form such that when the tasks exceeds the number of VMs the assignment starts over to the first sorted VM.

Algorithm 3: Pseudo-code of LJFP-PSO and MCT-PSO algorithms

```

1  Input:
2  No. of tasks ( $N_t$ ),
3  Task lengths ( $T$ ),
4  No. of VMs ( $N_v$ ),
5  VMs execution rates ( $V$ )
6  Output:
7  Allocation matrix (task to VM) of size  $N_t \times N_v$ 
8  Start:
9  Calculate the estimated execution time matrix of size
    $N_t \times N_v$ 
10 Initialize the allocation matrix (task to VM) of size  $N_t \times N_v$ 
   to zeros
11 Initialize PSO parameters
12 Initialize particles according to LJFP algorithm for LJFP-PSO,
   or
13 Initialize particles according to MCT algorithm for MCT-PSO.
14 For it = 1 to no. of iterations
15   For i = 1 to no. of particles
16     Calculate fitness value for each particle (MS, TET, DI, or
   TEC)
17     Find best particle so far.
18     Calculate the new velocity value according to equation
   (v)
19     Update the particles according to the maximum velocity
   value by
20     setting the corresponding VM location to 1 and 0 for the rest.
21   End
22   Find the global best particle
23 End
24 Set the allocation matrix to the global best particle

```

In algorithm 3, where the PSO is initialized heuristically, step 12 call algorithm 1 or 2 to initialize the PSO using MCT or LJFP algorithms respectively. The PSO iteration loop starts at step 14 and ends at 23. The algorithm processes each particle in the submain loop from steps 15 to 21. Within the submain loop, the fitness value for each particle (each solution represented by task to VM allocation matrix) is calculated. The fitness value depends on one of the four metrics

(MS, TET, DI, and TEC). The proposed algorithms will consider only one of these four metrics as an objective to find the optimum task scheduling. The local and global solutions along with their best fitness values are found in step 17. In steps 18 and 19, the velocity and position matrixes are updated according to Eqs. (10) and (9) respectively. At the end of the last main loop iteration, the optimal scheduling solution will be found in the global best particle allocation matrix.

5. Simulation results and discussion

In this section, the performance of the proposed algorithm is evaluated and compared with standard and swarm-based algorithms. The scheduling algorithms are simulated on MATLAB software installed on a PC with Intel® Core™ i7-5500U CPU with 2.40 GHz, and RAM of 8 GB running on 64-bit Windows 10 Pro operating system platform. The parameters of the virtual cloud computing environment and the PSO algorithm are defined in Tables 2 and 3 respectively.

The scheduling algorithms selected for comparison are; MCT, SJFP, LJFP, MinMin, MaxMin, PSO, and SJFP-PSO. Fig. 4, shows the simulation method structure. In addition, four recent metaheuristic algorithms (MALO, MSDE, WOA, and DSOS) are compared the proposed algorithms.

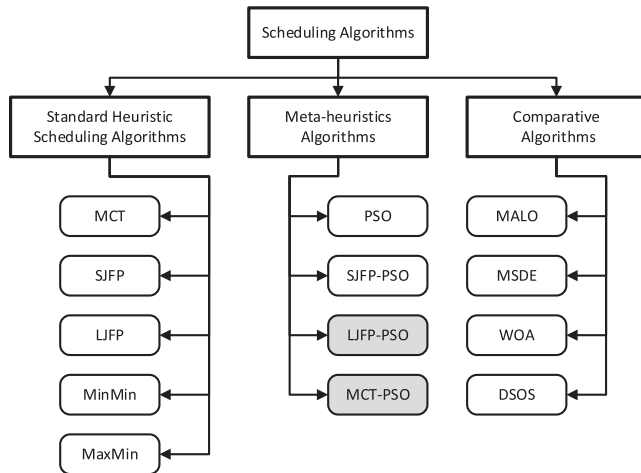
The performance is compared based on four main metrics; MS, TET, DI, and TEC. At first, for specific numbers of tasks and virtual machines, a Matlab program generates random task lengths and random VMs execution rates within the ranges (1000–4000) in MI and (1000–5000) in MIPS respectively. In addition, the power consumption of each VM is set randomly between (200–1000) watt. The random power consumption values are related to VMs execution rates, i.e., the more execution rate a VM has the more power it consumes. In the simulation, a VM can have one of two power consumption states, active and idle states. In active state, a VM consumes 100% of its total power consumption denoted by α as defined by equation (8). On the other hand, in an idle state

Table 2
Cloud system parameters.

Parameters	Values
Number of tasks	200–1000
Size of tasks (MI)	1000–4000
Number of VMs	40–200
Execution rate of VMs (MIPS)	1000–5000
Power consumption of VMs (Watt)	200–1000
Percentage of power consumption of idle to active states	0.6–0.7

Table 3
PSO parameters.

Parameter	Value
Number of Iterations	100
Population size	70
Velocity bounds	$[-1, 1]$
Inertia factor ω	0.9–0.2
Acceleration constants $c1$ and $c2$	2

**Fig. 4.** Simulation method structure.

a VM consumes 60–70% of its active state denoted by β . The percentage idle to active state power consumptions of all virtual machines are randomly set between this interval.

The results of performance evaluation are obtained based on two sets of simulation scenarios which are stated as follows:

Scenario 1: In this scenario, the number of tasks (N_t) is varied between 200 and 1000 task with a step of 200. The total number of virtual machines is fixed to 100. Figs. 5–8 show the bar charts performance comparison between the algorithms in term of MS, TET, DI, and TEC respectively. The corresponding numerical results for Figs. 5–8 are listed directly after each of them in Tables 4–7

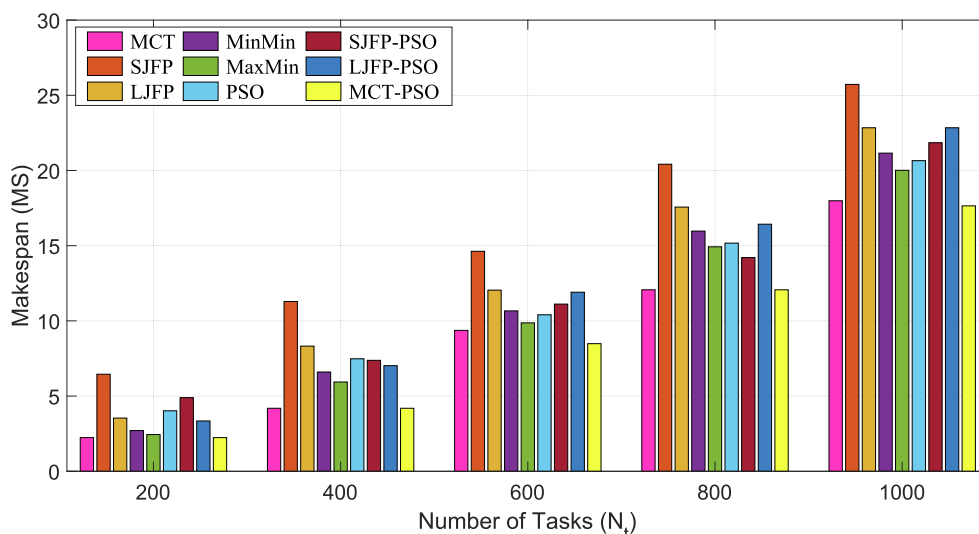
respectively. For the MS performance, As shown in Fig. 5, it can be observed that MS increases as the number of tasks increase for all algorithms. At all N_t values, the MCT-PSO achieved minimum MS. The SJFP algorithm ranked lastly having the highest MS values for all cases.

In Fig. 6, the algorithms are compared using TET. Similarly, for all algorithms, TET increases with the increase of tasks number. At $N_t = 200$, MCT-PSO and Min-Min have minimum TET. For the case at which $N_t = 400$, MCT-PSO reached the minimum. PSO recorded minimum TET values at 800 and 1000. Again, SJFP algorithm ranked lastly having the highest TET values for all cases. Also, LJFP ranked the second last among the algorithms.

Fig. 7 shows the performance comparison of the algorithms aiming to achieve load balancing. It can be observed that for small no of tasks, the difference between recorded DI values is high. As the task number increase, this difference decreases reaching almost comparable DI values at $N_t = 1000$. The MCT-PSO achieved the best balance between VMs at all N_t cases. Conversely, the SJFP has the worst load balance for all cases. Max-Min algorithm recorded minimum DI at $N_t = 200$. In all cases, SJFP has the highest DI values.

The performance comparison in terms of TEC is shown in Fig. 8. It is obvious that the energy consumption increases as the number of tasks increase. Compared to all algorithms, scheduling using MCT-PSO consumes the least energy for all cases. Just like the previous metrics, the maximum values of TEC is associated with SJFP for all cases.

Scenario 2: In this scenario, the number of tasks (N_t) is kept constant at 500. However, the number of virtual machines (N_v) varies between 40 and 200 VMs with a step of 40. Figs. 9–12 show the bar charts performance comparison between the algorithms in term of MS, TET, DI, and TEC respectively. In Fig. 9, it is expected that the makespan decreases as the number of machines increase. The MCT-PSO surpass all other algorithms for all cases except at $N_v = 40$ at which the PSO has minimum MS equal to 20.4 sec. It can also be observed that SJFP has the maximum MS values in all cases. Similarly, in Fig. 10, MCT-PSO has minimum TET values at $N_v = 120$ and 200. In terms of DI and TEC metrics, again, MCT-PSO achieved the best load balance and minimum energy consumption for most of the N_v cases as shown in Figs. 11 and 12 respectively. Tables 8–11 list the numerical results in scenario 2 for MS, TET, DI, and TEC respectively.

**Fig. 5.** Performance comparison between scheduling algorithms in scenario 1 using makespan (MS).

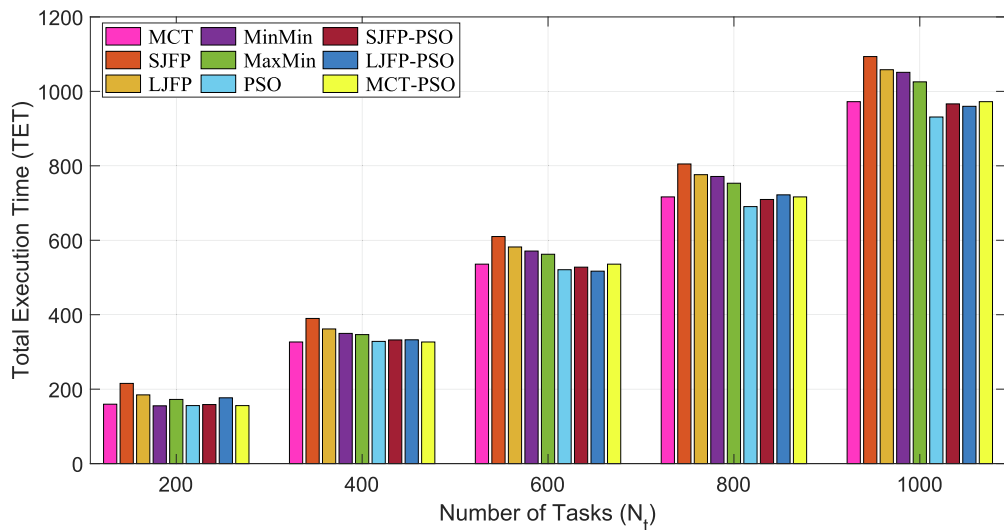


Fig. 6. Performance comparison between algorithms in scenario 1 using total execution time (TET).

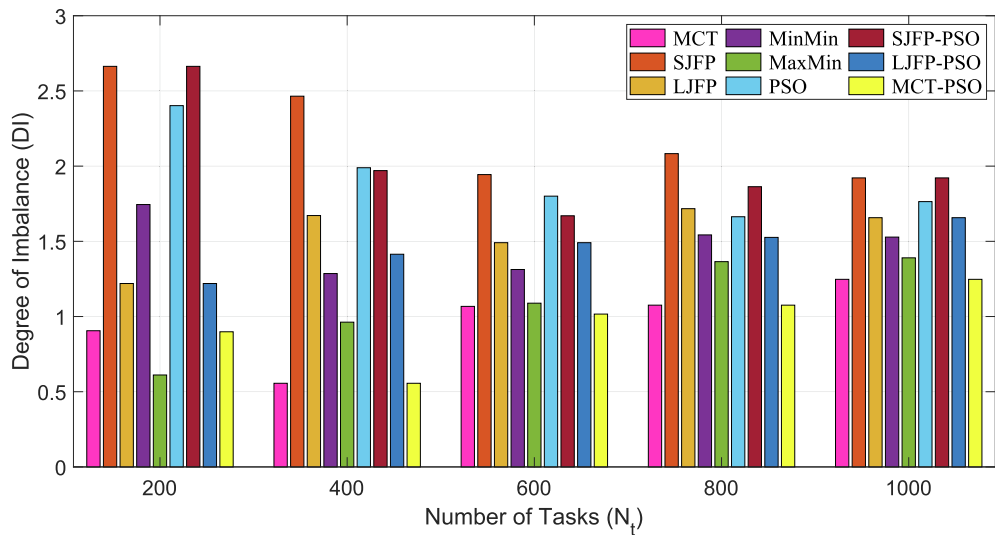


Fig. 7. Performance comparison between scheduling algorithms in scenario 1 using degree of imbalance (DI).

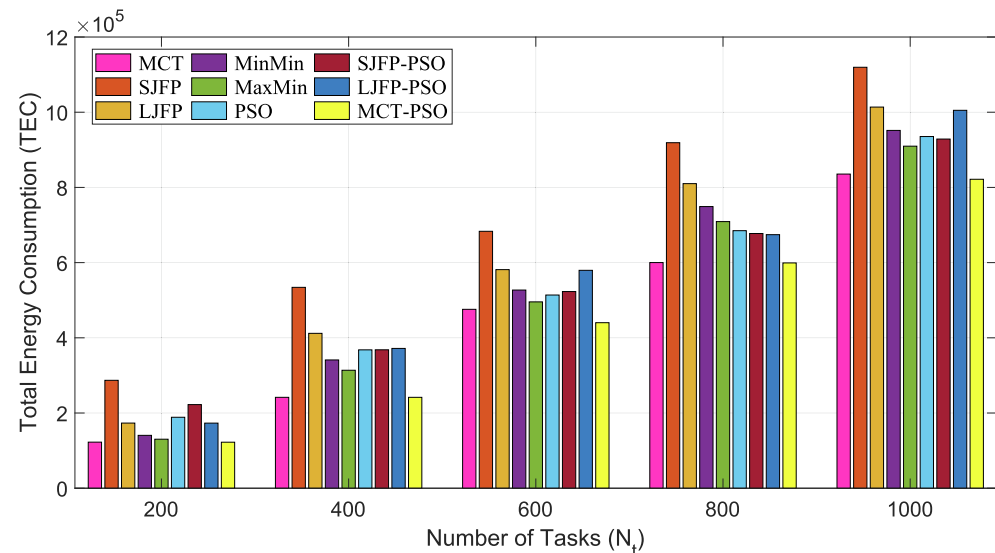


Fig. 8. Performance comparison between scheduling algorithms in scenario 1 using total energy consumption (TEC).

Table 4
Makespan performance comparison (scenario 1).

N_t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
200	2.2	6.5	3.5	2.7	2.4	4.0	4.9	3.3	2.2
400	4.2	11.3	8.3	6.6	5.9	7.5	7.4	7.0	4.2
600	9.4	14.6	12.0	10.7	9.9	10.4	11.1	11.9	8.5
800	12.1	20.4	17.6	16.0	14.9	15.2	14.2	16.4	12.1
1000	18.0	25.7	22.8	21.2	20.0	20.7	21.8	22.8	17.6

The optimal values are highlighted in bold.

Table 5
Total execution time performance comparison (scenario 1).

N_t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
200	159.6	215.5	184.6	155.3	172.4	155.9	158.7	176.8	155.8
400	326.7	390.0	361.7	349.8	346.6	328.4	332.2	332.6	326.7
600	535.8	610.0	582.0	571.0	562.4	520.9	527.8	517.0	535.8
800	716.5	805.0	776.2	771.4	753.2	690.4	709.6	722.0	716.5
1000	972.3	1093.5	1058.2	1051.2	1025.6	931.1	966.4	959.9	972.3

The optimal values are highlighted in bold.

Table 6
Degree of imbalance performance comparison (scenario 1).

N_t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
200	0.9	2.7	1.2	1.7	0.6	2.4	2.7	1.2	0.9
400	0.6	2.5	1.7	1.3	1.0	2.0	2.0	1.4	0.6
600	1.1	1.9	1.5	1.3	1.1	1.8	1.7	1.5	1.0
800	1.1	2.1	1.7	1.5	1.4	1.7	1.9	1.5	1.1
1000	1.2	1.9	1.7	1.5	1.4	1.8	1.9	1.7	1.2

The optimal values are highlighted in bold.

Table 7
Total energy consumption performance comparison (scenario 1).

N_t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
200	123	287	173	141	130	189	222	173	122
400	242	534	412	341	314	368	368	372	242
600	476	683	581	527	496	514	523	580	440
800	600	919	810	749	709	685	677	674	599
1000	836	1120	1014	952	910	935	929	1005	822

The optimal values are highlighted in bold.

The convergence curves of the PSO, SJFP-PSO, LJFP-PSO, and MCT-PSO are depicted in Fig. 13. In this experiment, the number of tasks and virtual machines are fixed to be 1000 and 20 respectively. The makespan as an objective is plotted with respect to the number of iterations. Only the first 20 iterations are shown in the plot as the convergence behavior of all algorithms is at steady after iteration 20.

As shown in Fig. 13, the PSO, SJFP-PSO, LJFP-PSO, and MCT-PSO converge to MS equal to 87, 81, 80, and 67 sec. respectively. The heuristic initialized PSO algorithms start with a MS values equal to their corresponding heuristic initialization. In this case, the SJFP-PSO, LJFP-PSO, and MCT-PSO start with 124, 121, and 114 respectively. Since the PSO initialize its particles randomly, it can start with any MS greater than or equal to its optimal value, here

it starts at MS = 100. As can be seen from Fig. 13, the MCT-PSO outperforms all other algorithms.

The performance of the proposed MCT-PSO and LJFP-PSO are compared with four well-known and recent metaheuristic algorithms. These comparative algorithms are; multi-objective antlion optimizer (MALO) (Abualigah and Diabat, 2020), moth search based on differential evolution (MSDE) (Elaziz et al., 2019), whale optimization algorithm (WOA) (Mirjalili and Lewis, 2016), and discrete symbiotic organism search algorithm (DSOS) (Abdullahi et al., 2016). In order to obtain fair comparison, the same simulation parameters used with the chosen algorithms are applied. To present the scalability of the proposed algorithms in the comparison, the number of tasks is varied from 100 to 1000 tasks with a step of 100. The tasks length

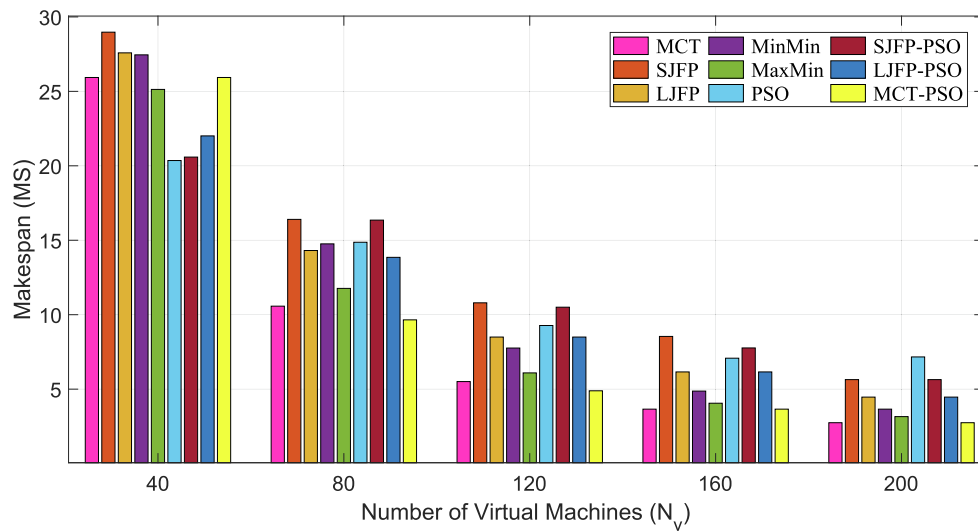


Fig. 9. Performance comparison between algorithms in scenario 2 using makespan (MS).

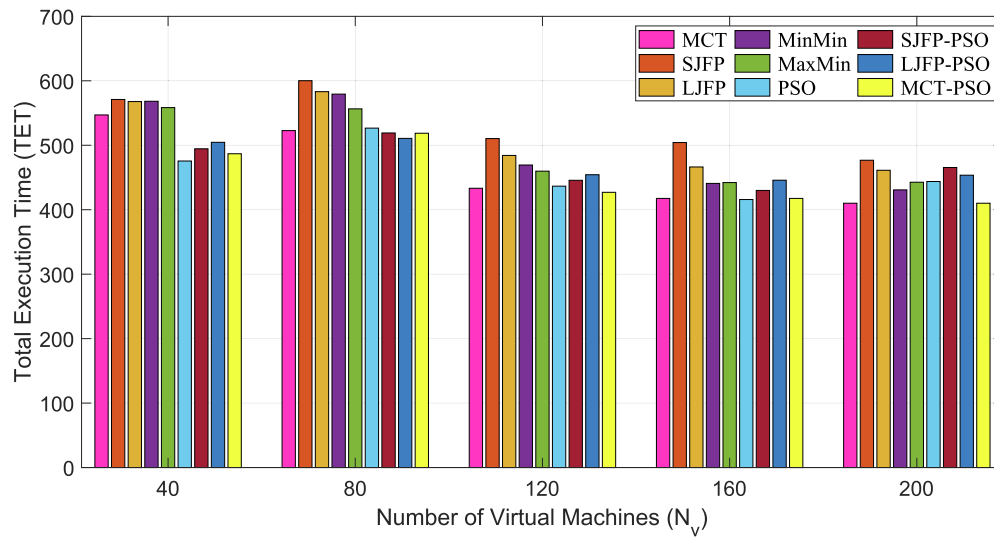


Fig. 10. Performance comparison between scheduling algorithms in scenario 2 using total execution time (TET).

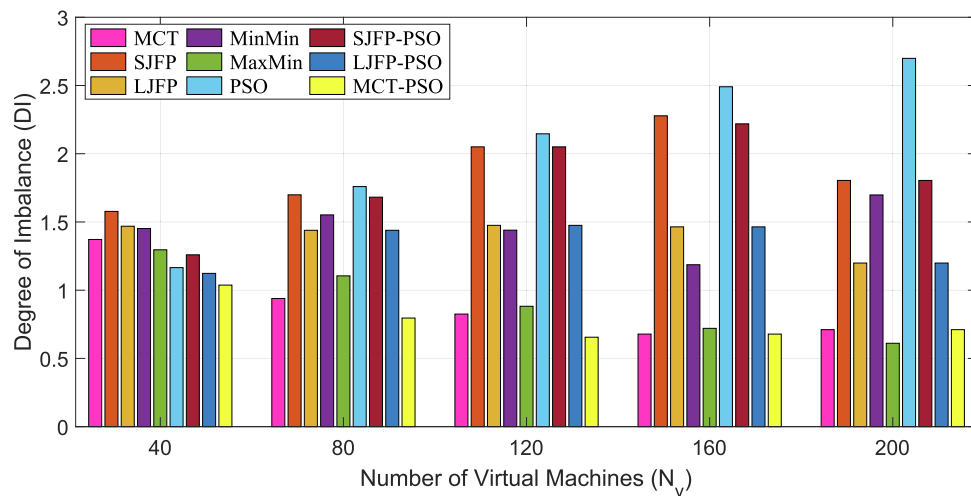


Fig. 11. Performance comparison between scheduling algorithms in scenario 2 using degree of imbalance (DI).

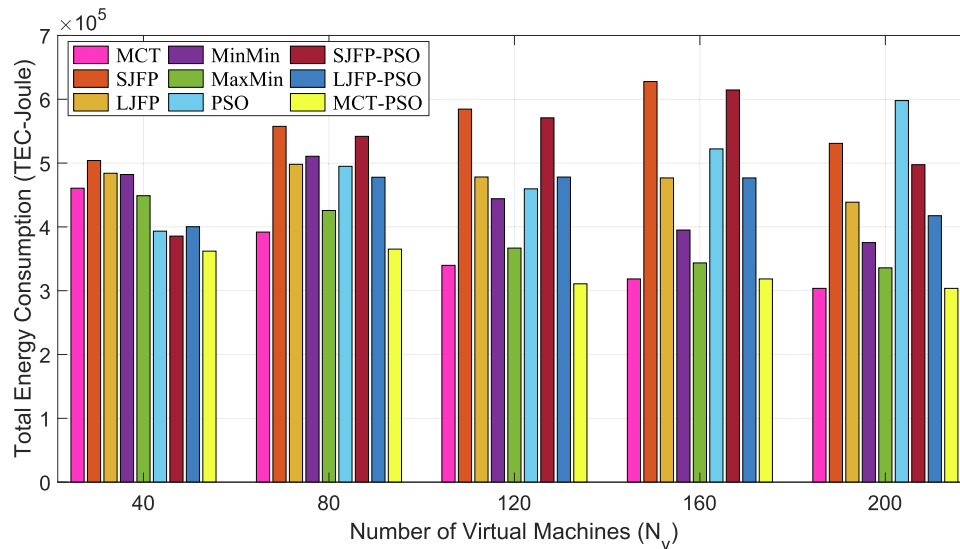


Fig. 12. Performance comparison between scheduling algorithms in scenario 2 using total energy consumption (TEC-Joule).

Table 8

Makespan performance comparison (scenario 2).

N _t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
40	25.9	29.0	27.6	27.5	25.1	20.4	20.6	22.0	25.9
80	10.6	16.4	14.3	14.8	11.8	14.9	16.4	13.9	9.7
120	5.5	10.8	8.5	7.8	6.1	9.3	10.5	8.5	4.9
160	3.7	8.5	6.2	4.9	4.1	7.1	7.8	6.2	3.7
200	2.7	5.6	4.5	3.7	3.2	7.2	5.6	4.5	2.7

The optimal values are highlighted in bold.

Table 9

Total execution time performance comparison (scenario 2).

N _t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
40	547.0	571.0	567.8	568.3	558.4	475.6	494.5	504.6	486.8
80	522.8	600.2	583.2	579.4	556.5	526.6	519.1	510.7	518.7
120	433.3	510.4	484.2	469.4	459.8	436.5	445.7	454.3	427.0
160	417.6	504.2	466.3	440.8	442.1	415.8	430.0	445.8	417.6
200	410.1	476.8	461.2	430.8	442.7	443.8	465.5	453.6	410.1

The optimal values are highlighted in bold.

Table 10

Degree of imbalance performance comparison (scenario 2).

N _t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
40	1.4	1.6	1.5	1.5	1.3	1.2	1.3	1.1	1.0
80	0.9	1.7	1.4	1.6	1.1	1.8	1.7	1.4	0.8
120	0.8	2.1	1.5	1.4	0.9	2.1	2.1	1.5	0.7
160	0.7	2.3	1.5	1.2	0.7	2.5	2.2	1.5	0.7
200	0.7	1.8	1.2	1.7	0.6	2.7	1.8	1.2	0.7

The optimal values are highlighted in bold.

defined by million instructions (MI) are generated uniformly between 1000 and 2000. Moreover, the number of virtual machines is kept constant at $N_v = 20$. The processing power of the VMs ranges from 1000 to 10,000 MIPS. Fig. 14 shows the

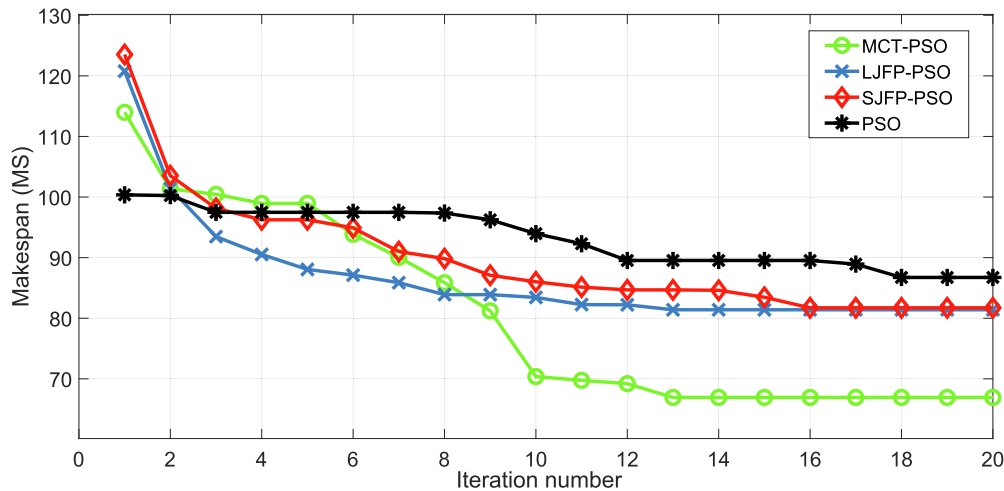
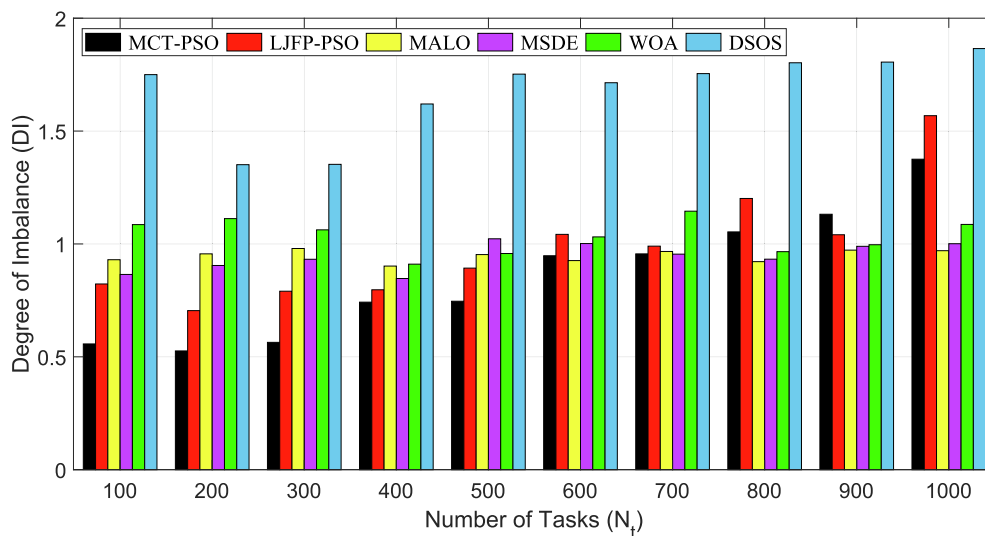
comparison results between the proposed and the comparative algorithms. The comparison is based on the performance of the algorithms in balancing the load among the virtual machines defined by DI.

Table 11

Total energy consumption performance comparison (scenario 2).

N_t	Algorithm								
	MCT	SJFP	LJFP	MinMin	MaxMin	PSO	SJFP-PSO	LJFP-PSO	MCT-PSO
40	461	504	484	482	449	393	386	400	362
80	392	558	498	511	426	495	542	478	365
120	340	585	478	444	367	460	571	478	311
160	318	628	477	395	344	522	615	477	318
200	304	531	439	375	336	598	497	418	304

The optimal values are highlighted in bold.

**Fig. 13.** Convergence behaviour comparison of the proposed algorithms.**Fig. 14.** Performance comparison between algorithms using degree of imbalance.

From Fig. 14, it can be observed that the MCT-PSO and LJFP-PSO attain optimum load balance by scoring minimum DI for $N_t = 100$ to 500. At $N_t = 600$ to 900 the proposed algorithms have almost comparable DI values to their comparatives. However,

only at high number of tasks ($N_t = 1000$), the comparative algorithms overcome the proposed algorithms and record less DI values except for DSOS algorithm. Table 12, summarize the numerical comparison results.

Table 12
Summary of numerical comparison results.

N_t	Algorithms					
	MCT-PSO	LJFP-PSO	MALO	MSDE	WOA	DSOS
100	0.5571	0.8227	0.9301	0.8649	1.0857	1.7500
200	0.5261	0.7047	0.9561	0.9046	1.1122	1.3511
300	0.5641	0.7906	0.9798	0.9321	1.0624	1.3526
400	0.7422	0.7969	0.9020	0.8469	0.9105	1.6200
500	0.7463	0.8929	0.9529	1.0228	0.9576	1.7522
600	0.9477	1.0424	0.9263	1.0017	1.0312	1.7141
700	0.9560	0.9903	0.9667	0.9548	1.1451	1.7545
800	1.0534	1.2015	0.9216	0.9325	0.9658	1.8025
900	1.1315	1.0407	0.9726	0.9895	0.9965	1.8055
1000	1.3754	1.5681	0.9701	1.0010	1.0865	1.8654

The optimal values are highlighted in bold.

6. Conclusion

In cloud computing, adopting an efficient task scheduling algorithm is vital for both users and providers. In this paper, heuristic algorithms are used to assist PSO algorithm for task scheduling. The particles of the PSO are heuristically initialized using longest job to fastest processor (LJFP) and minimum completion time (MCT) algorithms. Starting the search process with LJFP and MCT selected based solutions can significantly influence the convergence speed and performance. Simulation results demonstrate the effective performance of the proposed LJFP-PSO and MCT-PSO algorithms. Also, comparison simulation results revealed that the proposed heuristic initialized PSO outperforms recent task scheduling algorithms in terms of convergence and load balancing. It is worth noting that the proposed heuristic initialization of the PSO population creates initial particles that all start the search process from a single starting position. This starting point is created using a heuristic algorithm. As a future work, it is possible to investigate the use of different heuristic methods to create different starting search points for the PSO or any other metaheuristic algorithms.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Abdullahi, M., Ngadi, M.A., Abdulhamid, S.M., 2016. Symbiotic organism search optimization based task scheduling in cloud computing environment. *Future Generat. Comp. Syst.* 56, 640–650. <https://doi.org/10.1016/j.future.2015.08.006>.
 Abualigah, L., Diabat, A., 2020. A novel hybrid antlion optimization algorithm for multi-objective task scheduling problems in cloud computing environments. *Cluster Comput.* 5. <https://doi.org/10.1007/s10586-020-03075-5>.
 Adhikari, M., Nandy, S., Amgoth, T., 2019. Meta heuristic-based task deployment mechanism for load balancing in IaaS cloud. *J. Network Comp. Appl.* 128, 64–77. <https://doi.org/10.1016/j.jnca.2018.12.010>.
 Al-Arasi, R., Saif, A., 2020. Task scheduling in cloud computing based on metaheuristic techniques: A review paper. *EAI Endorsed Trans. Cloud Syst.* 6, 162829. <https://doi.org/10.4108/eai.13-7-2018.162829>.
 Arunarani, A.R., Manjula, D., Sugumaran, V., 2019. Task scheduling techniques in cloud computing: a literature survey. *Future Generat. Comp. Syst.* 91, 407–415. <https://doi.org/10.1016/j.future.2018.09.014>.

Belgacem, A., Beghdad-Bey, K., Nacer, H., Bouznad, S., 2020. Efficient dynamic resource allocation method for cloud computing environment. *Cluster Comput.* 23 (4), 2871–2889. <https://doi.org/10.1007/s10586-020-03053-x>.
 Chaudhary, D., Kumar, B., 2018. Cloudy GSA for load scheduling in cloud computing. *Appl. Soft Comput.* 71, 861–871. <https://doi.org/10.1016/j.asoc.2018.07.046>.
 Devaraj, A.F.S., Elhoseny, M., Dhanasekaran, S., Lydia, E.L., Shankar, K., 2020. Hybridization of firefly and improved multi-objective particle swarm optimization algorithm for energy efficient load balancing in cloud computing environments. *J. Parallel Distrib. Comput.* 142, 36–45. <https://doi.org/10.1016/j.jpdc.2020.03.022>.
 Elaziz, M.A., Xiong, S., Jayasena, K.P.N., Li, L., 2019. Task scheduling in cloud computing based on hybrid moth search algorithm and differential evolution. *Knowl.-Based Syst.* 169, 39–52. <https://doi.org/10.1016/j.knsys.2019.01.023>.
 Fanian, F., Bardsiri, V.K., Shokouhifar, M., 2018. A new task scheduling algorithm using firefly and simulated annealing algorithms in cloud computing. *Int. J. Adv. Comput. Sci. Appl.* 9, 195–202. <https://doi.org/10.14569/IJACSA.2018.090228>.
 Golchi, M.M., Saraeian, S., Heydari, M., 2019. A hybrid of firefly and improved particle swarm optimization algorithms for load balancing in cloud environments: performance evaluation. *Comput. Netw.* 162, 106860. <https://doi.org/10.1016/j.comnet.2019.106860>.
 Jena, U.K., Das, P.K., Kabat, M.R., 2020. Hybridization of meta-heuristic algorithm for load balancing in cloud computing environment. *J. King Saud Univ. – Comput. Inf. Sci.* 1–11. <https://doi.org/10.1016/j.jksuci.2020.01.012>.
 Kaur, A., Kaur, B., 2022. Load balancing optimization based on hybrid Heuristic-Metaheuristic techniques in cloud environment. *J. King Saud Univ. – Comp. Inf. Sci.* 34, 813–824. <https://doi.org/10.1016/j.jksuci.2019.02.010>.
 Keshanchi, B., Souri, A., Navimipour, N.J., 2017. An improved genetic algorithm for task scheduling in the cloud environments using the priority queues: formal verification, simulation, and statistical testing. *J. Syst. Softw.* 124, 1–21. <https://doi.org/10.1016/j.jss.2016.07.006>.
 Kumar, M., Sharma, S.C., 2018. PSO-COGEN: Cost and energy efficient scheduling in cloud environment with deadline constraint. *Sustainable Comput. Inf. Syst.* 19, 147–164. <https://doi.org/10.1016/j.suscom.2018.06.002>.
 Mahato, D.P., Singh, R.S., Tripathi, A.K., Maurya, A.K., 2017. On scheduling transactions in a grid processing system considering load through Ant Colony Optimization. *Appl. Soft Comput.* 61, 875–891. <https://doi.org/10.1016/j.asoc.2017.08.047>.
 Mansouri, N., Mohammad Hasani Zade, B., Javidi, M.M., 2019. Hybrid task scheduling strategy for cloud computing by modified particle swarm optimization and fuzzy theory. *Comput. Ind. Eng.* 130, 597–633. <https://doi.org/10.1016/j.cie.2019.03.006>.
 Mirjalili, S., Lewis, A., 2016. The whale optimization algorithm. *Adv. Eng. Softw.* 95, 51–67. <https://doi.org/10.1016/j.advengsoft.2016.01.008>.
 Mishra, S.K., Sahoo, B., Parida, P.P., 2020. Load balancing in cloud computing: A big picture. *J. King Saud Univ. – Comp. Inf. Sci.* 32 (2), 149–158. <https://doi.org/10.1016/j.jksuci.2018.01.003>.
 Sardaraz, M., Tahir, M., 2019. A hybrid algorithm for scheduling scientific workflows in cloud computing. *IEEE Access* 7, 186137–186146. <https://doi.org/10.1109/ACCESS.2019.2961106>.
 Strumberger, I., Bacanin, N., Tuba, M., Tuba, E., 2019. Resource scheduling in cloud computing based on a hybridized whale optimization algorithm. *Appl. Sci.* 9. <https://doi.org/10.3390/app9224893>.